

Socket Programming Practice Assignments

PCCSL507 – Computer Networks Laboratory

Department of Computer Science & Engineering

Submitted By

Ammu Vijayan

S5 CSE A

Roll No:11

Question 1 – Hello Client-Server (Beginner)

server.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <arpa/inet.h>
```

```
#include <sys/socket.h>
```

```
#define PORT 8080
```

```
int main()
```

```
{
```

```
    int serverSocket, clientSocket;
```

```
    struct sockaddr_in serverAddr;
```

```
    char buffer[1024];
```

```
    /* Create Socket */
```

```
    serverSocket = socket(AF_INET, SOCK_STREAM, 0);
```

```
    if (serverSocket < 0)
```

```
{
```

```

        perror("Socket Creation Failed");
        exit(1);
    }

    printf("Socket Created Successfully...\n");

    /* Server Address */
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(PORT);

    /* Bind */
    if (bind(serverSocket,
            (struct sockaddr *)&serverAddr,
            sizeof(serverAddr)) < 0)
    {
        perror("Bind Failed");
        exit(1);
    }

    printf("Bind Successful...\n");

    /* Listen */
    listen(serverSocket, 5);

    printf("Waiting for Client...\n");

    /* Accept */
    clientSocket = accept(serverSocket, NULL, NULL);

    if (clientSocket < 0)
    {

```

```

        perror("Accept Failed");
        exit(1);
    }

    printf("Client Connected...\n");

    /* Receive */
    recv(clientSocket, buffer, sizeof(buffer), 0);

    printf("Client Says: %s\n", buffer);

    /* Reply */
    strcpy(buffer, "Hello Client");

    send(clientSocket, buffer, strlen(buffer) + 1, 0);

    printf("Reply Sent...\n");

    /* Close */
    close(clientSocket);
    close(serverSocket);

    printf("Connection Closed...\n");

    return 0;
}

```

client.c

```

#include <stdio.h>
#include <stdlib.h>

```

```

#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

#define PORT 8080

int main()
{
    int sockfd;
    struct sockaddr_in serverAddr;
    char buffer[1024];

    /* Create Socket */
    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    if (sockfd < 0)
    {
        perror("Socket Creation Failed");
        exit(1);
    }

    printf("Socket Created Successfully...\n");

    /* Server Address */
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);

    inet_pton(AF_INET, "127.0.0.1", &serverAddr.sin_addr);

    /* Connect */
    if (connect(sockfd,

```

```

        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr)) < 0)
    {
        perror("Connection Failed");
        exit(1);
    }

    printf("Connected to Server...\n");

    /* Send Message */
    strcpy(buffer, "Hello Server");

    send(sockfd, buffer, strlen(buffer) + 1, 0);

    printf("Message Sent: Hello Server\n");

    /* Receive Reply */
    recv(sockfd, buffer, sizeof(buffer), 0);

    printf("Reply from Server: %s\n", buffer);

    /* Close */
    close(sockfd);

    printf("Connection Closed...\n");

    return 0;
}

```

Output:

server.c

```
student@LAPTOP-MCLNADP! x + v
Setting up libgcc-13-dev:amd64 (13.3.0-6ubuntu2-24.04.1) ...
Setting up libc6-dev:amd64 (2.39-0ubuntu8.8) ...
Setting up cpp-x86-64-linux-gnu (4:13.2.0-7ubuntu1) ...
Setting up cpp-13 (13.3.0-6ubuntu2-24.04.1) ...
Setting up gcc-13-x86-64-linux-gnu (13.3.0-6ubuntu2-24.04.1) ...
Setting up gcc-13 (13.3.0-6ubuntu2-24.04.1) ...
Setting up cpp (4:13.2.0-7ubuntu1) ...
Setting up gcc-x86-64-linux-gnu (4:13.2.0-7ubuntu1) ...
Setting up gcc (4:13.2.0-7ubuntu1) ...
Setting up libheif-plugin-aomdec:amd64 (1.17.6-1ubuntu4.6) ...
Setting up libheif1:amd64 (1.17.6-1ubuntu4.6) ...
Setting up libgd3:amd64 (2.3.3-9ubuntu5) ...
Setting up libc-devtools (2.39-0ubuntu8.8) ...
Setting up libheif-plugin-aomenc:amd64 (1.17.6-1ubuntu4.6) ...
Processing triggers for libc-bin (2.39-0ubuntu8.8) ...
Processing triggers for man-db (2.12.0-4build2) ...
student@LAPTOP-MCLNADP!~$ cd /mnt/c/Users/hp/Desktop/tcp
student@LAPTOP-MCLNADP!:/mnt/c/Users/hp/Desktop/tcp$ ls
client.c server.c
student@LAPTOP-MCLNADP!:/mnt/c/Users/hp/Desktop/tcp$ gcc server.c -o server
student@LAPTOP-MCLNADP!:/mnt/c/Users/hp/Desktop/tcp$ ./server
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Client Says: Hello Server
Reply Sent...
Connection Closed...
student@LAPTOP-MCLNADP!:/mnt/c/Users/hp/Desktop/tcp$ |
```

client.c

```
student@LAPTOP-MCLNADP! x + v
student@LAPTOP-MCLNADP!~$ cd /mnt/c/Users/hp/Desktop/tcp
student@LAPTOP-MCLNADP!:/mnt/c/Users/hp/Desktop/tcp$ ls
client.c server.c
student@LAPTOP-MCLNADP!:/mnt/c/Users/hp/Desktop/tcp$ gcc client.c -o client
student@LAPTOP-MCLNADP!:/mnt/c/Users/hp/Desktop/tcp$ ./client
Socket Created Successfully...
Connected to Server...
Message Sent: Hello Server
Reply from Server: Hello Client
Connection Closed...
student@LAPTOP-MCLNADP!:/mnt/c/Users/hp/Desktop/tcp$ |
```

Question 2 – Arithmetic Server

arithmetic_server.c.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <arpa/inet.h>
```

```

int main() {
    int server_fd, client_socket;
    int num1, num2;
    float results[4];

    struct sockaddr_in server, client;
    socklen_t client_len = sizeof(client);

    // Create socket
    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    if (server_fd < 0) {
        perror("Socket creation failed");
        return 1;
    }

    printf("Socket Created Successfully...\n");

    // Server address
    server.sin_family = AF_INET;
    server.sin_addr.s_addr = INADDR_ANY;
    server.sin_port = htons(8080);

    // Bind
    if (bind(server_fd, (struct sockaddr *)&server, sizeof(server)) < 0) {
        perror("Bind failed");
        return 1;
    }

    printf("Bind Successful...\n");

```

```
// Listen
listen(server_fd, 5);

printf("Waiting for Client...\n");

// Accept client
client_socket = accept(server_fd,
                      (struct sockaddr *)&client,
                      &client_len);

if (client_socket < 0) {
    perror("Accept failed");
    return 1;
}

printf("Client Connected...\n");

// Receive two integers
recv(client_socket, &num1, sizeof(num1), 0);
recv(client_socket, &num2, sizeof(num2), 0);

printf("Received Numbers: %d and %d\n", num1, num2);

// Arithmetic operations
results[0] = num1 + num2;
results[1] = num1 - num2;
results[2] = num1 * num2;

if (num2 != 0)
    results[3] = (float)num1 / num2;
else
    results[3] = 0;
```



```

// Send results
send(client_socket, results, sizeof(results), 0);

printf("Results Sent Successfully...\n");

// Close connections
close(client_socket);
close(server_fd);

printf("Connection Closed...\n");

return 0;
}

```

arithmetic_client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int client_socket;
    int num1, num2;
    float results[4];

    struct sockaddr_in server;

    // Create socket
    client_socket = socket(AF_INET, SOCK_STREAM, 0);

    if (client_socket < 0) {
        perror("Socket creation failed");
    }
}

```

```

    return 1;
}

printf("Socket Created Successfully...\n");

// Server address
server.sin_family = AF_INET;
server.sin_port = htons(8080);
server.sin_addr.s_addr = inet_addr("127.0.0.1");

// Connect to server
if (connect(client_socket, (struct sockaddr *)&server, sizeof(server)) < 0) {
    perror("Connection failed");
    return 1;
}

printf("Connected to Server...\n");

// Input two integers
printf("Enter first integer: ");
scanf("%d", &num1);

printf("Enter second integer: ");
scanf("%d", &num2);

// Send numbers
send(client_socket, &num1, sizeof(num1), 0);
send(client_socket, &num2, sizeof(num2), 0);

printf("Numbers Sent Successfully...\n");

// Receive results

```

```
recv(client_socket, results, sizeof(results), 0);

// Display results
printf("\nAddition = %.2f\n", results[0]);
printf("Subtraction = %.2f\n", results[1]);
printf("Multiplication = %.2f\n", results[2]);

if (num2 != 0)
    printf("Division = %.2f\n", results[3]);
else
    printf("Division = Cannot divide by zero\n");

// Close connection
close(client_socket);

printf("Connection Closed...\n");

return 0;
}
```

Output:

arithmetic_server.c.

```
student@LAPTOP-MCLNADP:~$ cd /mnt/c/Users/hp/Desktop/tcp
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ ls
client  client.c  server  server.c
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ nano arithmetic_server.c
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ ls
arithmetic_server.c  client  client.c  server  server.c
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ gcc arithmetic_server.c -o arithmetic_server
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ ./arithmetic_server
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received Numbers: 25 and 35
Results Sent Successfully...
Connection Closed...
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ |
```

arithmetic_client.c

```
student@LAPTOP-MCLNADP:~$ cd /mnt/c/Users/hp/Desktop/tcp
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ nano arithmetic_client.c
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ gcc arithmetic_client.c -o arithmetic_client
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ ./arithmetic_client
Socket Created Successfully...
Connected to Server...
Enter first integer: 25
Enter second integer: 35
Numbers Sent Successfully...

Addition = 60.00
Subtraction = -10.00
Multiplication = 875.00
Division = 0.71
Connection Closed...
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ |
```

Question 3 – String Processing Server

string_server.c

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <arpa/inet.h>
```

```
int main() {
    int server_fd, client_socket;
    char str[100];
    char uppercase[100];
    char reversed[100];
    int length;

    struct sockaddr_in server, client;
    socklen_t client_len = sizeof(client);

    server_fd = socket(AF_INET, SOCK_STREAM, 0);

    if (server_fd < 0) {
        perror("Socket creation failed");
        return 1;
    }

    printf("Socket Created Successfully...\n");

    server.sin_family = AF_INET;
    server.sin_addr.s_addr = INADDR_ANY;
    server.sin_port = htons(8080);

    if (bind(server_fd, (struct sockaddr *)&server, sizeof(server)) < 0) {
        perror("Bind failed");
        return 1;
    }

    printf("Bind Successful...\n");

    listen(server_fd, 5);
```

```
printf("Waiting for Client...\n");
```

```
client_socket = accept(server_fd,  
                        (struct sockaddr *)&client,  
                        &client_len);
```

```
if (client_socket < 0) {  
    perror("Accept failed");  
    return 1;  
}
```

```
printf("Client Connected...\n");
```

```
recv(client_socket, str, sizeof(str), 0);
```

```
str[strcspn(str, "\n")] = '\0';
```

```
printf("Received String: %s\n", str);
```

```
length = strlen(str);
```

```
strcpy(uppercase, str);
```

```
for (int i = 0; uppercase[i] != '\0'; i++) {  
    if (uppercase[i] >= 'a' && uppercase[i] <= 'z') {  
        uppercase[i] = uppercase[i] - 32;  
    }  
}
```

```
strcpy(reversed, str);
```

```

int n = strlen(reversed);

for (int i = 0; i < n / 2; i++) {
    char temp = reversed[i];
    reversed[i] = reversed[n - i - 1];
    reversed[n - i - 1] = temp;
}

send(client_socket, &length, sizeof(length), 0);
send(client_socket, uppercase, sizeof(uppercase), 0);
send(client_socket, reversed, sizeof(reversed), 0);

printf("Results Sent Successfully...\n");

close(client_socket);
close(server_fd);

printf("Connection Closed...\n");

return 0;
}

```

string_client.c.

```

#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int client_socket;
    char str[100];
    char uppercase[100];
    char reversed[100];

```

```
int length;

struct sockaddr_in server;

client_socket = socket(AF_INET, SOCK_STREAM, 0);

if (client_socket < 0) {
    perror("Socket creation failed");
    return 1;
}

printf("Socket Created Successfully...\n");

server.sin_family = AF_INET;
server.sin_port = htons(8080);
server.sin_addr.s_addr = inet_addr("127.0.0.1");

if (connect(client_socket, (struct sockaddr *)&server, sizeof(server)) < 0) {
    perror("Connection failed");
    return 1;
}

printf("Connected to Server...\n");

printf("Enter a string: ");
fgets(str, sizeof(str), stdin);

send(client_socket, str, sizeof(str), 0);

printf("String Sent Successfully...\n");

recv(client_socket, &length, sizeof(length), 0);
```



```

recv(client_socket, uppercase, sizeof(uppercase), 0);
recv(client_socket, reversed, sizeof(reversed), 0);

printf("\nString Length = %d\n", length);
printf("Uppercase = %s\n", uppercase);
printf("Reversed = %s\n", reversed);

close(client_socket);

printf("Connection Closed...\n");

return 0;
}

```

Output:

string_server.c

```

student@LAPTOP-MCLNADP: ~$ cd /mnt/c/Users/hp/Desktop/tcp
student@LAPTOP-MCLNADP: /mnt/c/Users/hp/Desktop/tcp$ nano string_server.c
student@LAPTOP-MCLNADP: /mnt/c/Users/hp/Desktop/tcp$ ls
arithmetic_client.c  arithmetic_server.c  client.c  server.c  string_server.c
student@LAPTOP-MCLNADP: /mnt/c/Users/hp/Desktop/tcp$ gcc string_server.c -o string_server
student@LAPTOP-MCLNADP: /mnt/c/Users/hp/Desktop/tcp$ ./string_server
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received String: Computer Networks
Results Sent Successfully...
Connection Closed...
student@LAPTOP-MCLNADP: /mnt/c/Users/hp/Desktop/tcp$ |

```

string_client.c

```
student@LAPTOP-MCLNADP: ~$ cd /mnt/c/Users/hp/Desktop/tcp
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ nano string_client.c
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ gcc string_client.c -o string_client
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ ./string_client
Socket Created Successfully...
Connected to Server...
Enter a string: Computer Networks
String Sent Successfully...

String Length = 17
Uppercase = COMPUTER NETWORKS
Reversed = skrowtEŃ retupmoC
Connection Closed...
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ |
```

Question 4 – Array Processing Server

array_server.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <arpa/inet.h>
```

```
int main() {
```

```
    int server_fd, client_socket;
```

```
    int n;
```

```
    int arr[100];
```

```
    int sum = 0;
```

```
    int max, min;
```

```
    float average;
```

```
    struct sockaddr_in server, client;
```

```
    socklen_t client_len = sizeof(client);
```

```
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
```

```
if (server_fd < 0) {
    perror("Socket creation failed");
    return 1;
}

printf("Socket Created Successfully...\n");

server.sin_family = AF_INET;
server.sin_addr.s_addr = INADDR_ANY;
server.sin_port = htons(8080);

if (bind(server_fd, (struct sockaddr *)&server, sizeof(server)) < 0) {
    perror("Bind failed");
    return 1;
}

printf("Bind Successful...\n");

listen(server_fd, 5);

printf("Waiting for Client...\n");

client_socket = accept(server_fd,
                      (struct sockaddr *)&client,
                      &client_len);

if (client_socket < 0) {
    perror("Accept failed");
    return 1;
}
```

```
printf("Client Connected...\n");
```

```
recv(client_socket, &n, sizeof(n), 0);
```

```
recv(client_socket, arr, sizeof(int) * n, 0);
```

```
printf("Received %d integers:\n", n);
```

```
for (int i = 0; i < n; i++) {
```

```
    printf("%d ", arr[i]);
```

```
    sum += arr[i];
```

```
}
```

```
printf("\n");
```

```
max = arr[0];
```

```
min = arr[0];
```

```
for (int i = 1; i < n; i++) {
```

```
    if (arr[i] > max)
```

```
        max = arr[i];
```

```
    if (arr[i] < min)
```

```
        min = arr[i];
```

```
}
```

```
average = (float)sum / n;
```

```
send(client_socket, &sum, sizeof(sum), 0);
```

```
send(client_socket, &average, sizeof(average), 0);
```

```
send(client_socket, &max, sizeof(max), 0);
```

```
send(client_socket, &min, sizeof(min), 0);
```

```
printf("Results Sent Successfully...\n");

close(client_socket);
close(server_fd);

printf("Connection Closed...\n");

return 0;
}
```

array_client.c.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <arpa/inet.h>

int main() {
    int client_socket;
    int n;
    int arr[100];
    int sum, max, min;
    float average;

    struct sockaddr_in server;

    client_socket = socket(AF_INET, SOCK_STREAM, 0);

    if (client_socket < 0) {
        perror("Socket creation failed");
        return 1;
    }

    printf("Socket Created Successfully...\n");
```

```

server.sin_family = AF_INET;
server.sin_port = htons(8080);
server.sin_addr.s_addr = inet_addr("127.0.0.1");

if (connect(client_socket, (struct sockaddr *)&server, sizeof(server)) < 0) {
    perror("Connection failed");
    return 1;
}

printf("Connected to Server...\n");

printf("Enter number of elements: ");
scanf("%d", &n);

printf("Enter %d integers:\n", n);

for (int i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
}

send(client_socket, &n, sizeof(n), 0);
send(client_socket, arr, sizeof(int) * n, 0);

printf("Array Sent Successfully...\n");

recv(client_socket, &sum, sizeof(sum), 0);
recv(client_socket, &average, sizeof(average), 0);
recv(client_socket, &max, sizeof(max), 0);
recv(client_socket, &min, sizeof(min), 0);

printf("\nSum = %d\n", sum);

```

```

printf("Average = %.2f\n", average);
printf("Maximum Element = %d\n", max);
printf("Minimum Element = %d\n", min);

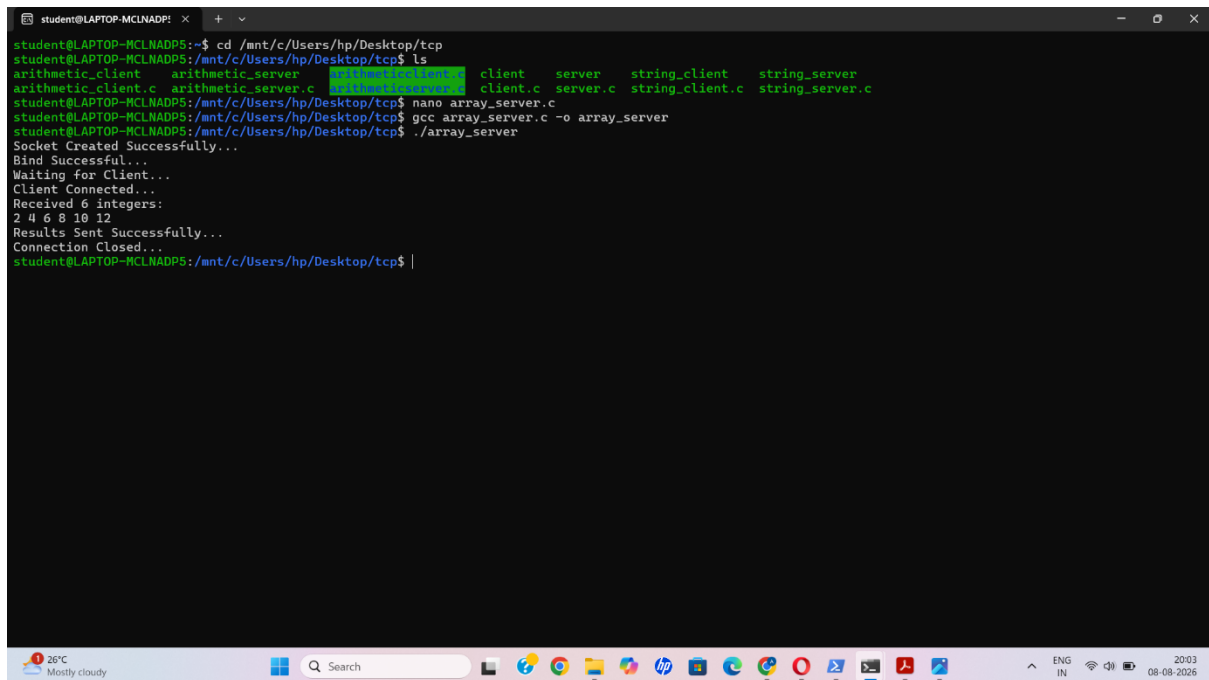
close(client_socket);

printf("Connection Closed...\n");

return 0;
}

```

array_server.c



```

student@LAPTOP-MCLNADP: ~$ cd /mnt/c/Users/hp/Desktop/tcp
student@LAPTOP-MCLNADP: /mnt/c/Users/hp/Desktop/tcp$ ls
arithmetic_client.c  arithmetic_server.c  array_server.c  client.c  server.c  string_client.c  string_server.c
student@LAPTOP-MCLNADP: /mnt/c/Users/hp/Desktop/tcp$ nano array_server.c
student@LAPTOP-MCLNADP: /mnt/c/Users/hp/Desktop/tcp$ gcc array_server.c -o array_server
student@LAPTOP-MCLNADP: /mnt/c/Users/hp/Desktop/tcp$ ./array_server
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received 6 integers:
2 4 6 8 10 12
Results Sent Successfully...
Connection Closed...
student@LAPTOP-MCLNADP: /mnt/c/Users/hp/Desktop/tcp$ |

```

array_client.c.

```
student@LAPTOP-MCLNADP: ~$ cd /mnt/c/Users/hp/Desktop/tcp
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ nano array_client.c
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ gcc array_client.c -o array_client
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ ./array_client
Socket Created Successfully...
Connected to Server...
Enter number of elements: 6
Enter 6 integers:
2 4 6 8 10 12
Array Sent Successfully...

Sum = 42
Average = 7.00
Maximum Element = 12
Minimum Element = 2
Connection Closed...
student@LAPTOP-MCLNADP:/mnt/c/Users/hp/Desktop/tcp$ |
```

Question 5 – Matrix Analyzer

matrix_server.c.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <arpa/inet.h>
```

```
struct MatrixData {
    int n;
    int matrix[10][10];
};
```

```
struct Result {
    int matrix_type;
    int sum;
    int trace;
};
```

```
int main() {
```



```
int server_fd, client_socket;
```

```
struct MatrixData data;
```

```
struct Result result;
```

```
struct sockaddr_in server, client;
```

```
socklen_t client_len = sizeof(client);
```

```
server_fd = socket(AF_INET, SOCK_STREAM, 0);
```

```
if (server_fd < 0) {  
    perror("Socket creation failed");  
    return 1;  
}
```

```
printf("Socket Created Successfully...\n");
```

```
server.sin_family = AF_INET;  
server.sin_addr.s_addr = INADDR_ANY;  
server.sin_port = htons(8080);
```

```
if (bind(server_fd, (struct sockaddr *)&server, sizeof(server)) < 0) {  
    perror("Bind failed");  
    return 1;  
}
```

```
printf("Bind Successful...\n");
```

```
listen(server_fd, 5);
```

```
printf("Waiting for Client...\n");
```

```

client_socket = accept(server_fd,
                        (struct sockaddr *)&client,
                        &client_len);

if (client_socket < 0) {
    perror("Accept failed");
    return 1;
}

printf("Client Connected...\n");

recv(client_socket, &data, sizeof(data), MSG_WAITALL);

printf("Received Matrix of Order %d:\n", data.n);

result.sum = 0;
result.trace = 0;

int diagonal = 1;
int upper = 1;
int lower = 1;

for (int i = 0; i < data.n; i++) {
    for (int j = 0; j < data.n; j++) {

        printf("%d ", data.matrix[i][j]);

        result.sum += data.matrix[i][j];

        if (i == j)
            result.trace += data.matrix[i][j];
    }
}

```

```

        if (i != j && data.matrix[i][j] != 0)
            diagonal = 0;

        if (i > j && data.matrix[i][j] != 0)
            upper = 0;

        if (i < j && data.matrix[i][j] != 0)
            lower = 0;
    }

    printf("\n");
}

if (diagonal == 1)
    result.matrix_type = 1;
else if (upper == 1)
    result.matrix_type = 2;
else if (lower == 1)
    result.matrix_type = 3;
else
    result.matrix_type = 4;

send(client_socket, &result, sizeof(result), 0);

printf("Results Sent Successfully...\n");

close(client_socket);
close(server_fd);

printf("Connection Closed...\n");

return 0;

```

```
}
```

matrix_client.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
include <unistd.h>
```

```
#include <arpa/inet.h>
```

```
struct MatrixData {
```

```
int n;
```

```
int matrix[10][10];
```

```
};
```

```
struct Result {
```

```
int matrix_type;
```

```
int sum;
```

```
int trace;
```

```
};
```

```
int main() {
```

```
int client_socket;
```

```
struct MatrixData data;
```

```
struct Result result;
```

```
...
```

```
struct sockaddr_in server;
```

```
client_socket = socket(AF_INET, SOCK_STREAM, 0);
```

```
if (client_socket < 0) {
```

```
    perror("Socket creation failed");
```

```
    return 1;
```

```
}
```

```
printf("Socket Created Successfully...\n");
```

```
server.sin_family = AF_INET;
```

```
server.sin_port = htons(8080);
```

```
server.sin_addr.s_addr = inet_addr("127.0.0.1");
```

```
if (connect(client_socket, (struct sockaddr *)&server, sizeof(server)) < 0) {
```

```
    perror("Connection failed");
```

```
    return 1;
```

```
}
```

```
printf("Connected to Server...\n");
```

```
printf("Enter order of square matrix: ");
```

```
scanf("%d", &data.n);
```

```
printf("Enter %d x %d matrix elements:\n", data.n, data.n);
```

```
for (int i = 0; i < data.n; i++) {
```

```
    for (int j = 0; j < data.n; j++) {
```

```
        scanf("%d", &data.matrix[i][j]);
```

```
    }
```

```
}
```

```
send(client_socket, &data, sizeof(data), 0);
```

```
printf("Matrix Sent Successfully...\n");
```

```
recv(client_socket, &result, sizeof(result), MSG_WAITALL);
```

```
printf("\nMatrix Type = ");
```

```

if (result.matrix_type == 1)
    printf("Diagonal Matrix\n");
else if (result.matrix_type == 2)
    printf("Upper Triangular Matrix\n");
else if (result.matrix_type == 3)
    printf("Lower Triangular Matrix\n");
else
    printf("General Matrix\n");

printf("Sum of Matrix Elements = %d\n", result.sum);
printf("Trace of Matrix = %d\n", result.trace);

close(client_socket);

printf("Connection Closed...\n");

return 0;

}

```

Output:

matrix_server.c.

```

student@LAPTOP-MCLNADP5:/mnt/c/Users/hp/Desktop/tcp$ nano matrix_server.c
student@LAPTOP-MCLNADP5:/mnt/c/Users/hp/Desktop/tcp$ gcc matrix_server.c -o matrix_server
student@LAPTOP-MCLNADP5:/mnt/c/Users/hp/Desktop/tcp$ ./matrix_server
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...
Received Matrix of Order 3:
1 2 3
0 4 5
0 0 6
Results Sent Successfully...
Connection Closed...
student@LAPTOP-MCLNADP5:/mnt/c/Users/hp/Desktop/tcp$ |

```

matrix_client.c

```
student@LAPTOP-MCLNADP5:/mnt/c/Users/hp/Desktop/tcp$ cd /mnt/c/Users/hp/Desktop/tcp
student@LAPTOP-MCLNADP5:/mnt/c/Users/hp/Desktop/tcp$ gcc matrix_client.c -o matrix_client
student@LAPTOP-MCLNADP5:/mnt/c/Users/hp/Desktop/tcp$ ./matrix_client
Socket Created Successfully...
Connected to Server...
Enter order of square matrix: 3
Enter 3 x 3 matrix elements:
1 2 3
0 4 5
0 0 6
Matrix Sent Successfully...

Matrix Type = Upper Triangular Matrix
Sum of Matrix Elements = 21
Trace of Matrix = 11
Connection Closed...
student@LAPTOP-MCLNADP5:/mnt/c/Users/hp/Desktop/tcp$ |
```